



Міністерство освіти і науки України
Національний технічний університет України “Київський політехнічний
інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE»,
«STRATEGY»»**
Система ведення нотаток

Виконав:
Студент групи ІА-24
Шкарніков А. С.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:	3
Мета:	3
Хід роботи	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми	3
2. Реалізувати один з розглянутих шаблонів за обраною темою	4
Висновки:	6
Код проекту:	6

Тема:

ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY»

Мета:

Ознайомитися з основними шаблонами проєктування, такими як «Singleton», «Iterator», «Proxy», «State» та «Strategy», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Завдання:

В якості нотаток має бути, як мінімум, текст або зображення, додатково можна додати підтримку файлів розміром до 2Мб, збереження посилань на веб-сторінки, збереження html-тексту з коректним його відображенням. Система повинна дозволяти вести нотатки онлайн, заводити блокноти, прикріплювати до нотаток мітки, давати можливість працювати в системі одночасно багатьом користувачам і одному користувачу працювати з різних комп'ютерів. Користувач повинен мати можливість дати доступ до свого блокноту з замітками іншому користувачу з різними правами (тільки перегляд; перегляд та редагування; перегляд, редагування, додавання та видалення нотаток)

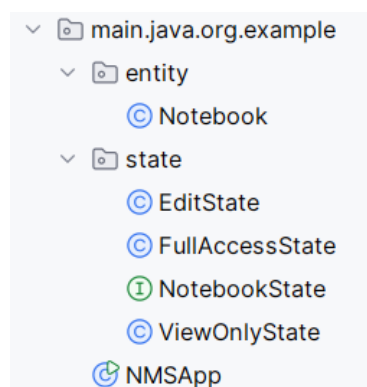
Хід роботи**1. Реалізувати не менше 3-х класів відповідно до обраної теми**

Рис. 1 — Структура проєкту

У процесі виконання лабораторної роботи було створено шість класів, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. Notebook

Клас відповідає за зберігання інформації про окрему пісню. Містить у собі такі властивості, як унікальний ідентифікатор (id), назва пісні (name), і URL-адреса

для доступу до файлу пісні (songUrl). Використовується для зберігання даних у плейлисті.

2. NotebookState

Основний клас для управління колекцією пісень. Реалізує функціонал для додавання, зберігання та видалення пісень із плейлиста. Крім того, надає метод createIterator(), який дозволяє створити конкретний ітератор (послідовний, перемішаний або повторюваний) для обходу списку пісень залежно від обраного режиму.

3. FullAccessState

Базовий клас-ітератор для послідовного обходу колекції пісень. Забезпечує методи hasNext() для перевірки наявності наступного елемента та next() для доступу до наступної пісні у плейлисті. Використовується як основа для створення спеціалізованих ітераторів.

4. EditState

Наслідує SongIterator та реалізує алгоритм для обходу пісень у випадковому порядку. При створенні об'єкта даного класу список пісень перемішується, а доступ до елементів здійснюється у випадковій послідовності. Застосовується для реалізації режиму відтворення "перемішування" (shuffle).

5. ViewOnlyState

Також наслідує SongIterator, але забезпечує зациклений обхід плейлиста. Після досягнення кінця списку відтворення повертається на початок. Цей ітератор використовується для реалізації режиму "повторення" (repeat).

2. Реалізувати один з розглянутих шаблонів за обраною темою

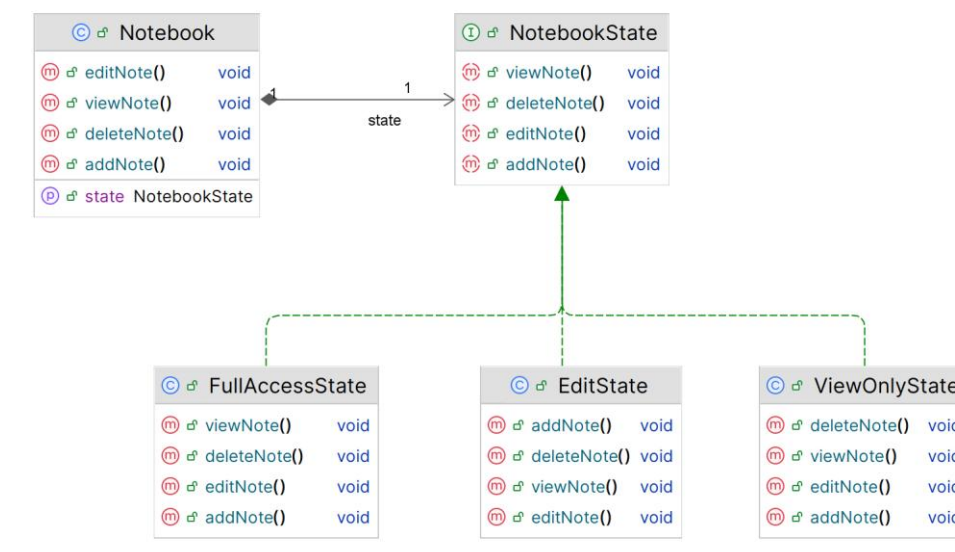


Рис. 2 — Діаграма класів

У системі ведення нотаток патерн State дозволяє легко керувати різними правами доступу до блокнотів, забезпечуючи чітке розмежування між функціоналом "лише перегляду", "перегляду та редагування", або "повного доступу". Завдяки цьому підходу система стає більш гнучкою, масштабованою та готовою до розширення, наприклад, для додавання нових типів доступу чи функцій.

Реалізація

Патерн **State** у системі ведення нотаток був реалізований через створення контекстного класу `Notebook`, який підтримує різні стани доступу до нотаток: повний доступ, доступ лише для редагування, або тільки перегляд. Для цього було створено базовий інтерфейс `NotebookState`, що визначає стандартні операції для роботи з нотатками (перегляд, редагування, додавання, видалення), і конкретні класи станів (`FullAccessState`, `EditState`, `ViewOnlyState`), які реалізують цей інтерфейс.

Контекстний клас `Notebook` керує поточним станом блокнота та делегує виконання операцій відповідному стану. Наприклад, якщо користувач має права лише на перегляд (`ViewOnlyState`), то методи додавання чи редагування нотаток будуть блокуватися й виводити відповідне повідомлення.

Проблеми, які вирішує патерн

- **Динамічна зміна поведінки.** Залежно від стану доступу користувача до блокнота (перегляд, редагування, повний доступ), змінюється поведінка програми без необхідності написання умовних конструкцій (наприклад, `if-else`). Система делегує виконання методів відповідному об'єкту стану, що дозволяє легко змінювати поведінку під час виконання програми.
- **Управління складністю.** У великих системах умовна логіка доступу може стати важкою для підтримки. Патерн State дозволяє розподілити логіку між кількома класами, кожен з яких відповідає за свій конкретний стан, спрощуючи підтримку та розширення.
- **Модульність і масштабованість.** Завдяки тому, що стани інкапсульовані у власних класах, додавання нового стану доступу (наприклад, лише коментування нотаток) стає простим завданням — потрібно лише створити новий клас, що реалізує інтерфейс `NotebookState`, і підключити його до контекстного класу.

Переваги використання

- **Зменшення кількості умов:** Замість перевірки прав доступу через численні if або switch конструкції, поведінка делегується об'єктам стану, що робить код чистішим і легшим для розуміння.
- **Гнучкість у зміні поведінки:** Контекстний клас може змінити стан у будь-який момент, забезпечуючи динамічну зміну поведінки програми залежно від ситуації (наприклад, оновлення прав користувача без перезавантаження).
- **Простота розширення:** Додавання нових типів доступу не впливає на існуючий код, що відповідає принципу відкритості/закритості (ОСР) з SOLID. Це знижує ризик внесення помилок при масштабуванні системи.
- **Легкість тестування:** Кожен стан має чітко визначену логіку, яку можна протестувати окремо, підвищуючи якість і надійність системи.

Висновки:

У ході лабораторної роботи було реалізовано патерн State для роботи з доступом користувачів до блокнотів. Використання цього шаблону дозволило підвищити гнучкість та модульність коду, що значно полегшує його підтримку та розширення.

Код проекту:

https://github.com/Atl4sDev/TRPZ_labs